

Redes Neuronales II

Backpropagation, funciones de activación y clasificación binaria y multiclase con NumPy,
TensorFlow + Keras y PyTorch

Documento técnico

Estudiante	Jahir Ricardo Avila Gutiérrez
Asignatura	Deep Learning
Docente	Nidia Stella Garcia Roa
Repositorio	https://github.com/jahirravila/redes-neuronales-II
Fecha	26 de septiembre de 2026

Contenido

1. Introducción y objetivos	3
2. Marco conceptual.....	3
2.1 Del perceptrón a la red multicapa	3
2.2 Funciones de activación.....	3
2.3 El algoritmo de backpropagation	4
2.4 Clasificación binaria vs. multiclase.....	5
2.5 Entrenamiento supervisado.....	5
2.6 Frameworks: TensorFlow + Keras y PyTorch	5
3. Metodología y entorno de trabajo	6
4. Perceptrón, red de una capa y red multicapa (NumPy).....	7
4.1 Funciones de activación.....	7
4.2 Perceptrón	7
4.3 Red neuronal de una capa	7
4.4 Red neuronal multicapa con backpropagation	8
4.5 Resultados	10
5. Clasificación binaria: Breast Cancer Wisconsin	11
5.1 Rediseño del entrenamiento: early stopping y validación cruzada.....	12
5.2 Evaluación final en test	13
5.3 Análisis.....	13
6. Clasificación multiclase en MNIST con TensorFlow + Keras	14
6.1 Resultados	15
7. Clasificación multiclase en MNIST con PyTorch.....	16
7.1 Resultados y comparación entre frameworks.....	17
8. Evidencias de ejecución.....	19
9. Análisis comparativo y conclusiones.....	24
10. Repositorio GitHub.....	24
Referencias	25

1. Introducción y objetivos

Este documento presenta el desarrollo de la actividad *Redes Neuronales II*. A partir de los modelos base del curso (perceptrón, red de una capa y red multicapa) se implementan desde cero, en NumPy, las funciones de activación Sigmoides y ReLU y el algoritmo de **backpropagation**; los modelos se rediseñan y optimizan para un problema real de **clasificación binaria** y, finalmente, se construye un clasificador **multiclase** de dígitos manuscritos (MNIST) con **TensorFlow + Keras** y se replica en **PyTorch**. De acuerdo con las condiciones de la actividad, **no se utiliza ninguna capa convolucional**: todos los modelos son redes densas (totalmente conectadas).

Objetivos específicos

- Implementar el perceptrón de Rosenblatt, una red de una capa (neurona sigmoide) y un perceptrón multicapa (MLP) con backpropagation, verificando numéricamente el gradiente.
- Comparar las funciones de activación Sigmoides y ReLU en términos de velocidad de convergencia y robustez.
- Aplicar y optimizar los modelos en un dataset real de clasificación binaria (*Breast Cancer Wisconsin*) mediante estandarización, validación cruzada, regularización L2 y *early stopping*.
- Implementar un MLP multiclase para MNIST con TensorFlow + Keras y replicar la misma arquitectura en PyTorch.
- Documentar los resultados, las evidencias de ejecución y publicar el código en GitHub.

Tabla 1. Notebooks desarrollados (todos ejecutables en Google Colab).

Notebook	Contenido	Herramientas
01_perceptron_capas_backprop	Activaciones, perceptrón, red de una capa, MLP + backpropagation, gradient checking, XOR y make_moons	NumPy
02_clasificacion_binaria_breast_cancer	Clasificación binaria con datos reales, búsqueda de hiperparámetros con CV, early stopping, métricas	NumPy, scikit-learn, Keras
03_mnist_tensorflow_keras	MLP multiclase 784-256-128-10 para MNIST	TensorFlow + Keras
04_mnist_pytorch	Réplica del MLP en PyTorch y comparación entre frameworks	PyTorch

2. Marco conceptual

2.1 Del perceptrón a la red multicapa

El **perceptrón** (Rosenblatt, 1958) calcula una combinación lineal de las entradas y aplica una función escalón. Su regla de aprendizaje corrige los pesos sólo cuando la muestra está mal clasificada:

$$\hat{y} = H(\mathbf{w}^\top \mathbf{x} + b), \quad \mathbf{w} \leftarrow \mathbf{w} + \eta (y - \hat{y}) \mathbf{x}, \quad b \leftarrow b + \eta (y - \hat{y})$$

El teorema de convergencia del perceptrón garantiza que el algoritmo termina si los datos son linealmente separables; en caso contrario (p. ej. XOR) oscila indefinidamente. Una **red de una capa** con activación sigmoide sigue siendo un clasificador lineal, pero es *diferenciable*, lo que permite minimizar una función de pérdida continua (entropía cruzada) por descenso de gradiente. El **perceptrón multicapa** añade capas ocultas con activaciones no lineales; por el teorema de aproximación universal, una capa oculta suficientemente ancha puede aproximar cualquier función continua, lo que permite fronteras de decisión no lineales.

2.2 Funciones de activación

Tabla 2. Funciones de activación implementadas.

Función	Definición	Derivada	Rango	Uso típico
Escalón	$H(z) = 1$ si $z \geq 0$; 0 si no	0 (no diferenciable en 0)	{0, 1}	Perceptrón clásico

Sigmoide	$\sigma(z) = 1 / (1 + e^{-z})$	$\sigma(z)(1 - \sigma(z)) \leq 0.25$	$(0, 1)$	Salida binaria (probabilidad)
Tanh	$\tanh(z)$	$1 - \tanh^2(z)$	$(-1, 1)$	Capas ocultas (centrada en 0)
ReLU	$\max(0, z)$	1 si $z > 0$; 0 si no	$[0, \infty)$	Capas ocultas en redes profundas
Softmax	$\exp(z_j) / \sum_j \exp(z_j)$	$s_k (\delta - s_k)$	$(0, 1), \sum = 1$	Salida multiclase

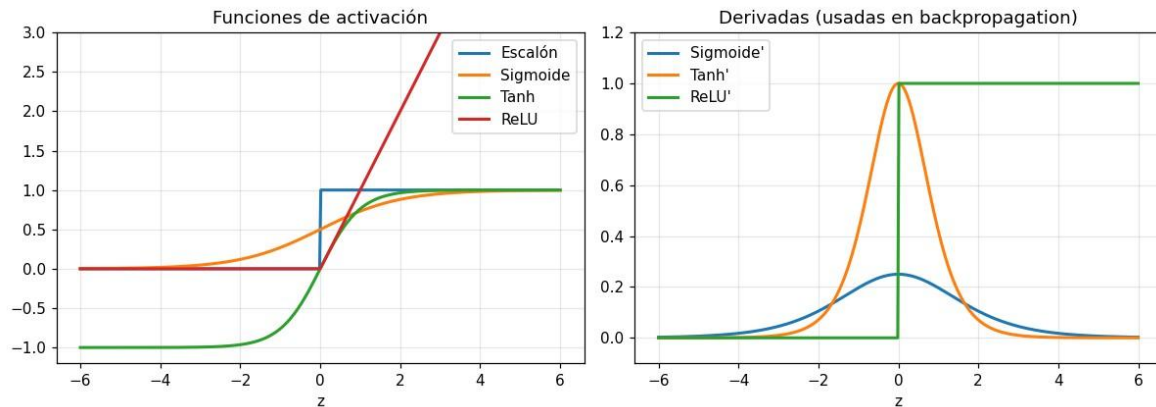


Figura 1. Funciones de activación (izquierda) y sus derivadas (derecha). La derivada de la sigmoide nunca supera 0.25, mientras que la de ReLU vale 1 en la zona activa.

La elección de la activación afecta directamente al flujo del gradiente. Como $\sigma'(z) \leq 0.25$, al atravesar L capas sigmoides el gradiente puede reducirse en un factor de hasta 0.25^L ($\approx 10^{-6}$ para 10 capas): es el problema del **desvanecimiento del gradiente**. ReLU no satura en la zona positiva y es computacionalmente trivial, por lo que es la activación oculta por defecto; la sigmoide se conserva en la salida binaria porque produce una probabilidad, y la softmax en la salida multiclase.

2.3 El algoritmo de backpropagation

Backpropagation (Rumelhart, Hinton y Williams, 1986) calcula eficientemente el gradiente de la pérdida respecto a todos los pesos aplicando la regla de la cadena desde la salida hacia la entrada. Para una red de L capas, el **paso hacia adelante** calcula y almacena:

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}, \quad \mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]}), \quad \mathbf{A}^{[0]} = \mathbf{X}$$

Con salida sigmoide + entropía cruzada binaria, o softmax + entropía cruzada categórica, el error de la última capa se simplifica a la diferencia entre predicción y etiqueta. El **paso hacia atrás** propaga ese error capa a capa multiplicando por la derivada de la activación:

$$\delta^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y} \quad \delta^{[l-1]} = (\delta^{[l]} \mathbf{W}^{[l]\top}) \odot g'^{[l-1]}(\mathbf{Z}^{[l-1]})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{1}{m} \mathbf{A}^{[l-1]\top} \delta^{[l]} + \frac{\lambda}{m} \mathbf{W}^{[l]}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{1}{m} \sum_i \delta_i^{[l]}$$

Finalmente, los pesos se actualizan con descenso de gradiente estocástico con *momentum* (en la implementación NumPy) o con Adam (en Keras y PyTorch):

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \eta \nabla_{\mathbf{W}} \mathcal{L}, \quad \mathbf{W} \leftarrow \mathbf{W} + \mathbf{v} \quad (\beta = 0.9)$$

Para comprobar que las derivadas están bien implementadas se utiliza **gradient checking**: se compara el gradiente analítico con la aproximación por diferencias centrales; un error relativo inferior a 10^{-7} indica una implementación correcta.

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon}, \quad \text{err}_{rel} = \frac{\|g_{an} - g_{num}\|}{\|g_{an}\| + \|g_{num}\|}$$

2.4 Clasificación binaria vs. multiclase

Tabla 3. Diferencias entre clasificación binaria y multiclase.

Aspecto	Binaria	Multiclase (K clases)
Codificación de y	0 / 1	Entero 0...K-1 o vector one-hot
Capa de salida	1 neurona, sigmoide	K neuronas, softmax
Pérdida	Entropía cruzada binaria (BCE)	Entropía cruzada categórica (CE)
Decisión	$\hat{y} \geq \text{umbral (0.5 por defecto)}$	$\underset{k}{\operatorname{argmax}} \hat{y}_k$
Métricas	Exactitud, precisión, recall, F1, ROC-AUC	Exactitud, matriz de confusión KxK, precisión/recall por clase, promedio macro
Casos en esta actividad	Breast Cancer (maligno / benigno)	MNIST (dígitos 0-9)

$$\mathcal{L}_{BCE} = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)], \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{m} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y})$$

$$\mathcal{L}_{CE} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}, \quad \hat{y}_{ik} = \operatorname{softmax}(\mathbf{z}_i)_k = \frac{e^{z_{ik}}}{\sum_j e^{z_{ij}}}$$

2.5 Entrenamiento supervisado

En el aprendizaje supervisado el modelo aprende una función $f(\mathbf{x}) \approx \mathbf{y}$ a partir de pares $(\mathbf{x}, \mathbf{y})$ etiquetados, minimizando una pérdida que mide la discrepancia entre predicción y etiqueta. Su correcta aplicación exige: (i) separar los datos en **entrenamiento**, **validación** (para ajustar hiperparámetros y decidir cuándo parar) y **test** (usado una sola vez para estimar el desempeño real); (ii) ajustar el preprocesamiento sólo con los datos de entrenamiento para evitar fugas de información; y (iii) controlar el sobreajuste con regularización (L2, Dropout) y *early stopping*.

2.6 Frameworks: TensorFlow + Keras y PyTorch

Tabla 4. Comparación conceptual de los frameworks utilizados.

Aspecto	TensorFlow + Keras	PyTorch
Definición del modelo	<code>keras.Sequential([...])</code> / API funcional	<code>nn.Sequential(...)</code> / subclase de <code>nn.Module</code>
Entrenamiento	<code>compile()</code> + <code>fit()</code> : bucle implícito	Bucle explícito: <code>forward</code> → <code>loss</code> → <code>backward</code> → <code>step</code>
Backpropagation	Automática dentro de <code>fit()</code> (GradientTape)	<code>loss.backward()</code> (autograd)
Salida multiclase	Capa <code>softmax</code> + <code>sparse_categorical_crossentropy</code>	Logits + <code>nn.CrossEntropyLoss</code> (log-softmax interno)
Dropout	Se desactiva automáticamente en <code>predict/evaluate</code>	<code>model.train()</code> / <code>model.eval()</code>
Fortaleza	Prototipado rápido, poco código	Control fino y transparencia del proceso

3. Metodología y entorno de trabajo

Los cuatro notebooks se desarrollaron para **Google Colab** (Python 3, sin dependencias adicionales: Colab ya incluye NumPy, scikit-learn, TensorFlow y PyTorch) y se ejecutan de principio a fin con *Entorno de ejecución* → *Ejecutar todas*. Todos los modelos funcionan en CPU; si hay GPU, PyTorch la detecta automáticamente. Las semillas aleatorias se fijan (SEED = 42) para que los resultados sean reproducibles.

Tabla 5. Entorno y configuración experimental.

Elemento	Detalle
Lenguaje / entorno	Python 3.11 · Google Colab / Jupyter
Librerías	NumPy 2.4 · pandas 3.0 · scikit-learn 1.8 · Matplotlib 3.10 · TensorFlow 2.21 (Keras 3.15) · PyTorch 2.14 / torchvision 0.29
Dataset binario	Breast Cancer Wisconsin (Diagnostic): 569 muestras, 30 variables, 212 malignos / 357 benignos (sklearn.datasets)
Dataset multiclase	MNIST: 60 000 imágenes de entrenamiento + 10 000 de test, 28×28 píxeles en escala de grises, 10 clases
Particiones	Breast Cancer: 60 % train / 20 % validación / 20 % test, estratificadas · MNIST: 54 000 train / 6 000 validación / 10 000 test
Preprocesamiento	Breast Cancer: StandardScaler ajustado sólo con train · MNIST: aplanado a 784 y normalización a [0, 1]
Restricción	Sin capas convolucionales: sólo capas densas

4. Perceptrón, red de una capa y red multicapa(NumPy)

Los tres modelos se implementaron desde cero con NumPy en el Notebook 01 y se reutilizan en el Notebook 02 (también disponibles como módulo `src/nn_numpy.py` en el repositorio). A continuación se presenta el código fuente completo de cada uno.

4.1 Funciones de activación

Código: funciones de activación y sus derivadas

```
def step(z):          return (z >= 0).astype(float)
def sigmoid(z):       return 1.0 / (1.0 + np.exp(-np.clip(z, -500,
500)))
def d_sigmoid(z):     s = sigmoid(z); return s * (1 - s)
def tanh(z):          return np.tanh(z)
def d_tanh(z):        return 1 - np.tanh(z) ** 2
def relu(z):          return np.maximum(0, z)
def d_relu(z):        return (z > 0).astype(float)
def softmax(z):       e = np.exp(z - z.max(axis=1, keepdims=True)) # estabilidad numérica
return e / e.sum(axis=1, keepdims=True)

ACTIVATIONS = {"sigmoid": (sigmoid, d_sigmoid), "tanh": (tanh, d_tanh), "relu": (relu, d_relu)}
```

4.2 Perceptrón

Código: clase Perceptron (regla de Rosenblatt)

```
class Perceptron:     """Perceptrón de Rosenblatt con activación escalón."""
def __init__(self, lr=0.1, epochs=50, seed=SEED):
    self.lr, self.epochs, self.rng = lr, epochs, np.random.default_rng(seed)

    def fit(self, X, y):
        self.w = self.rng.normal(0, 0.01, X.shape[1]); self.b = 0.0
        for _ in range(self.epochs):
            errors = 0
            for i in range(len(X)):
                update = self.lr * (y[i] - self.predict(X[i:i+1])[0])
                self.w += update * X[i]; self.b += update
            errors += int(update != 0)
            self.errors_.append(errors)
            if errors == 0:
                break
        return self

    def decision_function(self, X):
        return X @ self.w + self.b
    def predict(self, X):
        return step(self.decision_function(X))
    def score(self, X, y):
        return (self.predict(X) == y).mean()
```

4.3 Red neuronal de una capa

Código: clase SingleLayerNet (neurona sigmoide + BCE + descenso de gradiente)

```
class SingleLayerNet: """Red de una capa: una neurona sigmoide + entropía cruzada + descenso
de gradiente (batch)."""
def __init__(self, lr=0.5, epochs=2000, l2=0.0, seed=SEED):
    self.lr, self.epochs, self.l2, self.rng = lr, epochs, l2, np.random.default_rng(seed)

    def fit(self, X, y):
        m = len(X)
        self.w = self.rng.normal(0, 0.01, X.shape[1]); self.b = 0.0
        self.loss_ = []
        for _ in range(self.epochs):
            p = sigmoid(X @ self.w + self.b) # forward
            self.loss_.append(bce(y, p) + 0.5 * self.l2 * np.sum(self.w**2) / m)
            dz = (p - y) / m # backward
            self.w -= self.lr * (X.T @ dz + self.l2 * self.w / m)
            self.b -= self.lr * dz.sum()
        return self

    def predict_proba(self, X):
        return sigmoid(X @ self.w + self.b)
    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(float)
    def score(self, X, y):
        return (self.predict(X) == y).mean()

def bce(y, p, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

4.4 Red neuronal multicapa con backpropagation

La clase `MLP` admite cualquier número de capas y las activaciones ocultas Sigmoide, Tanh o ReLU. Respecto a la versión básica vista en semanas anteriores se **rediseñó y optimizó** con: inicialización He (ReLU) / Xavier (sigmoide, tanh), entrenamiento por mini-lotes, SGD con momentum, regularización L2, salida sigmoide o softmax según el número de neuronas de salida, y registro del historial de entrenamiento y validación. Los métodos `forward` y `backward` implementan literalmente las ecuaciones de la sección 2.3.

Código: clase MLP (forward, backward, actualización y entrenamiento)

```

class MLP:
    """Perceptrón multicapa en NumPy con
    backpropagation.

    layers: p. ej. [n_features, 16, 8, 1]. Salida sigmoide si la última capa tiene 1 neurona
    (clasificación binaria), softmax si tiene >1 (multiclase).
    """
    def __init__(self, layers, hidden="relu", lr=0.05, epochs=500, batch_size=32,
momentum=0.9, l2=0.0, seed=SEED, verbose=0):
        self.layers, self.hidden = layers, hidden
        self.g, self.dg = ACTIVATIONS[hidden]
        self.lr, self.epochs, self.bs = lr, epochs, batch_size
        self.beta, self.l2, self.verbose = momentum, l2, verbose
        self.rng = np.random.default_rng(seed)
        self.binary = layers[-1] == 1
        self._init_params()

    def _init_params(self):
        self.W, self.b = [], []
        for n_in, n_out in zip(self.layers[:-1], self.layers[1:]):
            scale = np.sqrt(2.0 / n_in) if self.hidden == "relu" else np.sqrt(1.0 / n_in) # He / Xavier
            self.W.append(self.rng.normal(0, scale, (n_in, n_out)))
            self.b.append(np.zeros((1, n_out)))
            self.vW = [np.zeros_like(w) for w in self.W]
            self.vb = [np.zeros_like(b) for b in self.b]

    # ----- forward -----
    def forward(self, X):
        A, cache = X, [(None, X)]
        for l, (W, b) in enumerate(zip(self.W, self.b)):
            Z = A @ W + b
            last = l == len(self.W) - 1
            A = (sigmoid(Z) if self.binary else softmax(Z)) if last else
            self.g(Z)
            cache.append((Z, A))
        return A, cache

    def loss(self, Y, A):
        data = bce(Y, A) if self.binary else -np.mean(np.sum(Y * np.log(np.clip(A, 1e-12, 1)),
axis=1))
        reg = 0.5 * self.l2 * sum(np.sum(W**2) for W in self.W) / len(Y)
        return data + reg

    # ----- backward -----
    def backward(self, Y, cache):
        m = len(Y)
        gW, gb = [None] * len(self.W), [None] * len(self.W)
        delta = cache[-1][1] - Y # δ^L
        for l in reversed(range(len(self.W))):
            A_prev = cache[l][1]
            gW[l] = A_prev.T @ delta / m + self.l2 * self.W[l] / m
            gb[l] = delta.sum(axis=0, keepdims=True) / m if l > 0:
            delta = (delta @ self.W[l].T) * self.dg(cache[l][0]) # δ^{l-1}
            return gW, gb

    def _step(self, gW, gb):
        for l in range(len(self.W)):
            self.vW[l] = self.beta * self.vW[l] - self.lr * gW[l]; self.W[l] += self.vW[l]
            self.vb[l] = self.beta * self.vb[l] - self.lr * gb[l]; self.b[l] += self.vb[l]

    def _prep_y(self, y):
        y = np.asarray(y)
        if self.binary: return y.reshape(-1, 1).astype(float)
        return np.eye(self.layers[-1])[y.astype(int)]

    def fit(self, X, y, X_val=None, y_val=None):
        Y = self._prep_y(y)
        self.history = {"loss": [], "acc": [], "val_loss": [], "val_acc": []}
        for ep in range(self.epochs):
            idx = self.rng.permutation(len(X))
            for s in range(0, len(X), self.bs):
                bi = idx[s:s + self.bs]
                A, cache = self.forward(X[bi])
                self._step(*self.backward(Y[bi], cache))
                A, _ = self.forward(X)
                self.history["loss"].append(self.loss(Y, A)); self.history["acc"].append(self.score(X, y))
            if X_val is not None:
                Av, _ = self.forward(X_val)
                self.history["val_loss"].append(self.loss(self._prep_y(y_val),
                self.history["val_acc"].append(self.score(X_val, y_val))
            if self.verbose and (ep + 1) % self.verbose == 0:
                msg = f"época {ep+1:4d} | pérdida {self.history['loss'][-1]:.4f} | acc

```

```

        {self.history['acc'][-1]:.3f}"
    if X_val is not None: msg += f" | val_pérdida {self.history['val_loss'][-1]:.4f} | val_acc
        {self.history['val_acc'][-1]:.3f}"
    print(msg)
    return

def predict_proba(self, X): return self.forward(X)[0]
def predict(self, X):
    A = self.predict_proba(X)
    return (A[:, 0] >= 0.5).astype(float) if self.binary else A.argmax(axis=1)
def score(self, X, y): return (self.predict(X) == np.asarray(y)).mean()

```

Código: verificación numérica del gradiente

```

def gradient_check(net, X, y, eps=1e-5):
    Y = net._prep_y(y)
    _, cache = net.forward(X)    gW,
    _ = net.backward(Y, cache)
    analytic, numeric = [], []
    for l,
W in enumerate(net.W):
        for idx
in np.ndindex(W.shape):
            old = W[idx]
            W[idx] = old + eps; lp = net.loss(Y, net.forward(X)[0])
            W[idx] = old - eps; lm = net.loss(Y, net.forward(X)[0])
            W[idx] = old
            numeric.append((lp - lm) / (2 * eps)); analytic.append(gW[l][idx])
a, n = np.array(analytic), np.array(numeric)
return np.linalg.norm(a - n) / (np.linalg.norm(a) + np.linalg.norm(n))

rng = np.random.default_rng(0) Xg =
rng.normal(size=(20, 3)) for hidden in
["sigmoid", "tanh", "relu"]:
    net_b = MLP([3, 5, 4, 1], hidden=hidden, l2=0.1); yb = rng.integers(0, 2, 20)    net_m
= MLP([3, 5, 4, 3], hidden=hidden, l2=0.1); ym = rng.integers(0, 3, 20)
print(f"{hidden:>7} | binaria: {gradient_check(net_b, Xg, yb):.2e} | multiclase (softmax):
    {gradient_check(net_m, Xg, ym):.2e}")

```

4.5 Resultados

Problemas lógicos AND, OR y XOR

Tabla 6. Exactitud sobre las tablas de verdad.

Modelo	AND	OR	XOR	Observación
Perceptrón	1.00 (5 épocas)	1.00 (4 épocas)	0.50	No converge en XOR (100 épocas)
Red de una capa	1.00	1.00	0.50	Pérdida en XOR estancada en $\ln 2 \approx 0.693$, predice 0.5
MLP 2-4-1 (tanh)	—	—	1.00	$p(y=1) = [0.002, 0.995, 0.997, 0.006]$

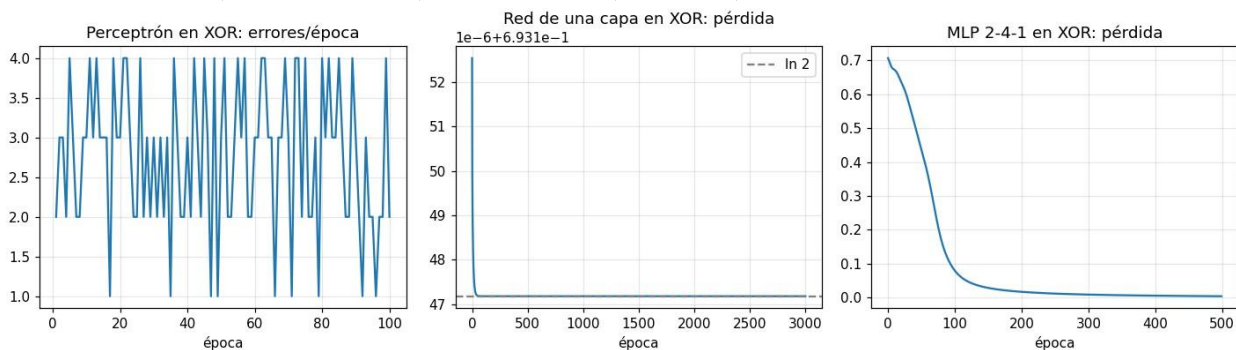


Figura 2. Convergencia en XOR: el perceptrón no logra cero errores, la red de una capa se estanca en $\ln 2$ y el MLP reduce la pérdida a casi cero.

Tabla 7. Gradient checking de la red 3-5-4-{1,3} con $L2 = 0.1$: el gradiente de backpropagation coincide con el numérico.

Activación oculta	Error relativo — salida binaria (sigmoide)	Error relativo — salida multiclase (softmax)
Sigmoide	8.95×10^{-11}	1.15×10^{-10}
Tanh	7.01×10^{-11}	1.85×10^{-10}

ReLU	3.77×10^{-11}	5.59×10^{-11}
------	------------------------	------------------------

Problema no lineal make_moons: Sigmoid vs ReLU

Con 600 puntos (ruido 0.2; 70 % train / 30 % test) se comparan los modelos lineales con dos MLP idénticos (2-16-16-1) que sólo difieren en la activación oculta.

Tabla 8. Resultados en make_moons.

Modelo	Exactitud train	Exactitud test	Épocas hasta val_acc $\geq$ 0.95
Perceptrón	0.883	0.867	—
Red de una capa (sigmoide)	0.871	0.867	—
MLP 2-16-16-1, oculta Sigmoide	0.962	0.967	161
MLP 2-16-16-1, oculta ReLU	0.964	0.978	19

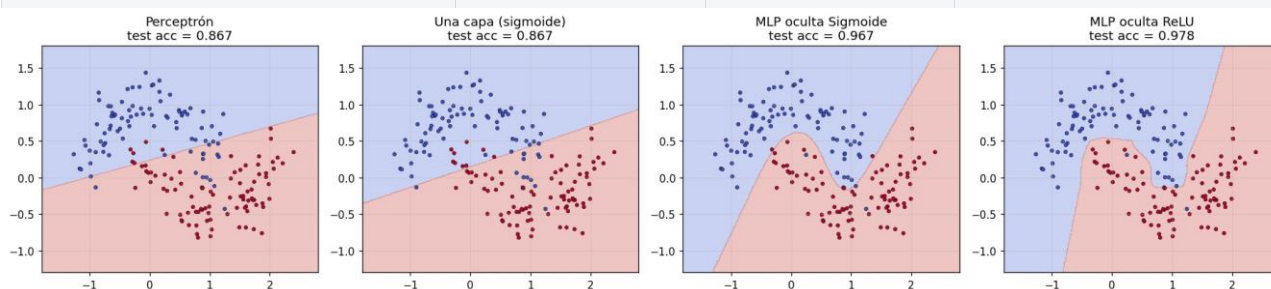


Figura 3. Fronteras de decisión: los modelos lineales sólo trazan una recta; el MLP aprende una frontera curva.

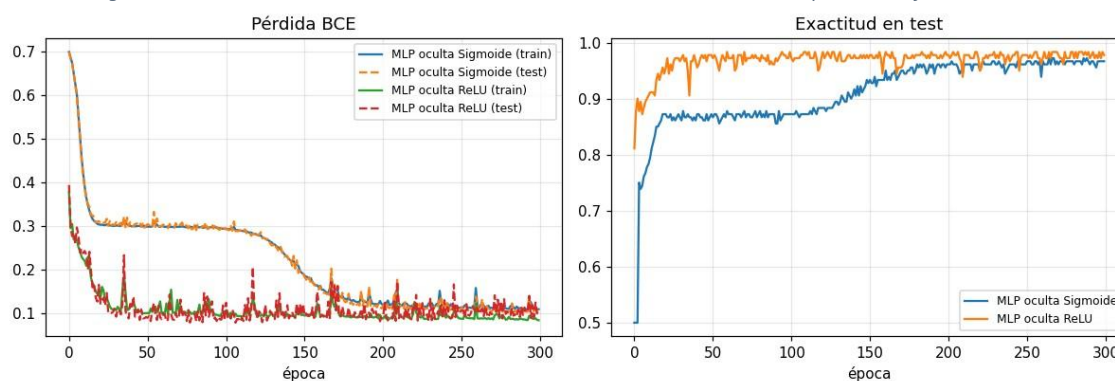


Figura 4. Curvas de aprendizaje del MLP con activación oculta Sigmoide y ReLU.

Interpretación. El perceptrón y la red de una capa son clasificadores lineales y fallan en XOR y en make_moons. Las capas ocultas no lineales resuelven ambos problemas. Con la misma arquitectura y tasa de aprendizaje, **ReLU alcanza el 95 % de exactitud en 19 épocas frente a 161 de la sigmoide**, lo que confirma empíricamente el efecto de la saturación de la sigmoide sobre el gradiente.

5. Clasificación binaria: Breast Cancer Wisconsin

El objetivo es predecir si un tumor es **maligno (0)** o **benigno (1)** a partir de 30 medidas de núcleos celulares (radio, textura, perímetro, área, concavidad, etc.). Las variables tienen escalas muy distintas (el área es del orden de 10^3 y la suavidad de 10^{-1}), por lo que se estandarizan.

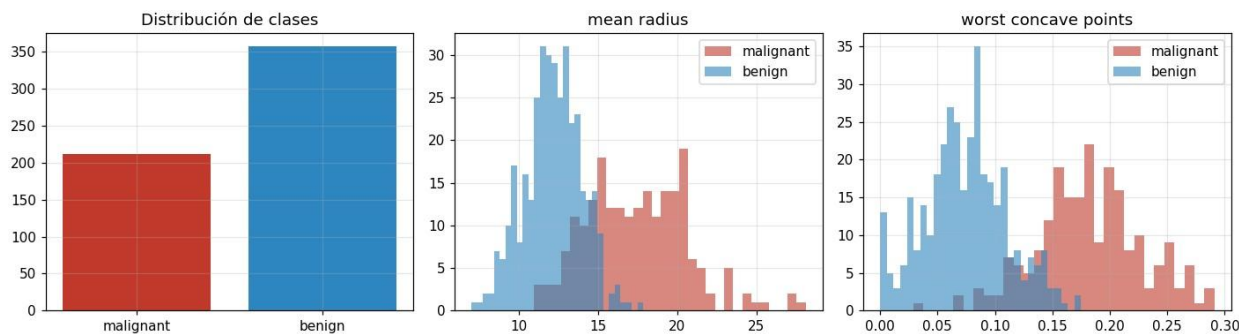


Figura 5. Distribución de clases y de dos variables discriminantes.

5.1 Rediseño del entrenamiento: early stopping y validación cruzada

Una primera comparación sin regularización mostró que el MLP con ReLU converge en pocas épocas pero sobreajusta (la pérdida de validación vuelve a subir), mientras que la sigmoide pasa varias épocas estancada en la clase mayoritaria (≈ 0.62).

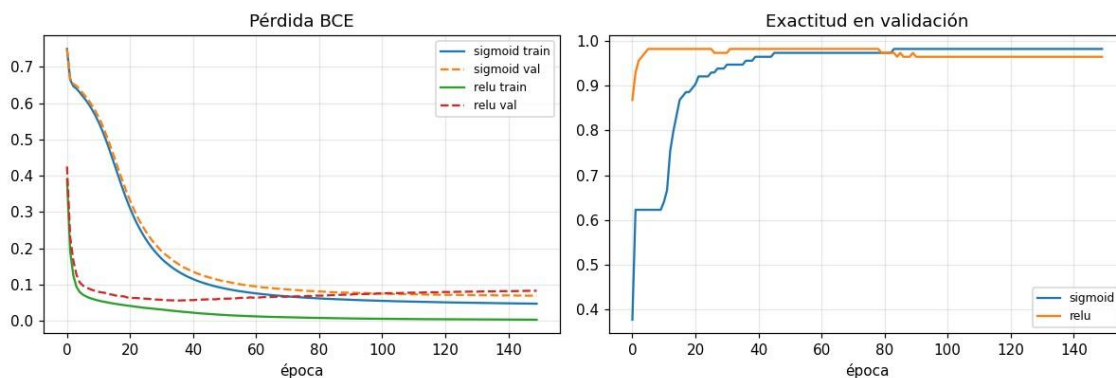


Figura 6. MLP 30-16-8-1 sin regularización: Sigmoide vs ReLU.

Para optimizar el modelo se extendió la clase MLP con **parada temprana** (restaura los pesos de la época con menor pérdida de validación) y se realizó una búsqueda en rejilla de 24 configuraciones con **validación cruzada estratificada de 5 particiones** (120 entrenamientos), ajustando el escalador dentro de cada partición:

Código: MLP con early stopping (extensión de la clase MLP)

```
class MLPEarlyStopping(MLP):
    def __init__(self, *args, patience=20, **kw):
        super().__init__(*args, **kw)
        self.patience = patience

    def fit(self, X, y, X_val, y_val):
        Y, Yv = self._prep_y(y), self._prep_y(y_val)
        self.history = {"loss": [], "acc": [], "val_loss": [], "val_acc": []}
        best, wait = np.inf, 0
        for ep in range(self.epochs):
            idx = self.rng.permutation(len(X))
            for s in range(0, len(X), self.bs):
                bi = idx[s:s + self.bs]; A, cache = self.forward(X[bi]); self._step(*self.backward(Y[bi], cache))
            vl = self.loss(Yv, self.forward(X_val)[0])
            self.history["loss"].append(self.loss(Y, self.forward(X)[0]));
            self.history["acc"].append(self.score(X, y))
            self.history["val_loss"].append(vl); self.history["val_acc"].append(self.score(X_val, y_val))
            if vl < best - 1e-5:
                best, wait, self.best_epoch = vl, 0, ep + 1
                self._best = ([w.copy() for w in self.W], [b.copy() for b in self.b])
            else:
                wait += 1
                if wait >= self.patience:
                    break
            self.W, self.b = self._best
        return self
```

Código: rejilla de hiperparámetros (extracto)

```
grid = {"hidden_layers": [(16,), (32, 16), (64, 32)], "activation": ["sigmoid", "relu"],
        "lr": [0.005, 0.02], "l2": [0.0, 5.0]}
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED) # ... para cada configuración y cada fold:
StandardScaler en el fold de train, entrenar, guardar min(val_loss)
```

Tabla 9. Mejores 5 configuraciones de la validación cruzada (ordenadas por pérdida media).

#	Capas ocultas	Activación	η	L2	Épocas (media)	Pérdida CV	Exactitud CV
1	(16,)	ReLU	0.02	0	102.0	0.0570	0.9824
2	(64, 32)	Sigmoide	0.02	0	121.0	0.0622	0.9780
3	(64, 32)	ReLU	0.005	0	122.2	0.0645	0.9780
4	(32, 16)	ReLU	0.02	0	28.4	0.0647	0.9846
5	(64, 32)	Sigmoide	0.005	0	236.6	0.0663	0.9758

Tabla 10. Robustez de cada activación sobre las 12 configuraciones en que se usa.

Activación	Pérdida CV media	Exactitud CV media	Exactitud CV mínima	Exactitud CV máxima
ReLU	0.1535	0.9694	0.9516	0.9846
Sigmoide	0.3322	0.8436	0.6264	0.9780

5.2 Evaluación final en test

Los modelos se evalúan una única vez en el conjunto de test (114 muestras). El recall de la clase maligna se reporta aparte porque un falso negativo (tumor maligno clasificado como benigno) es el error más costoso en este contexto.

Tabla 11. Métricas en test (clase positiva = benigno).

Modelo	Exactitud	Precisión	Recall	F1	Recall malignos	ROC-AUC
Perceptrón (NumPy)	0.9474	0.9583	0.9583	0.9583	0.9286	0.9861
Red de una capa (NumPy)	0.9825	0.9861	0.9861	0.9861	0.9762	0.9964
MLP 30-16-1 ReLU optimizado (NumPy)	0.9649	0.9722	0.9722	0.9722	0.9524	0.9884
MLP 30-16-1 ReLU (Keras, control)	0.9386	0.9851	0.9167	0.9496	0.9762	0.9924

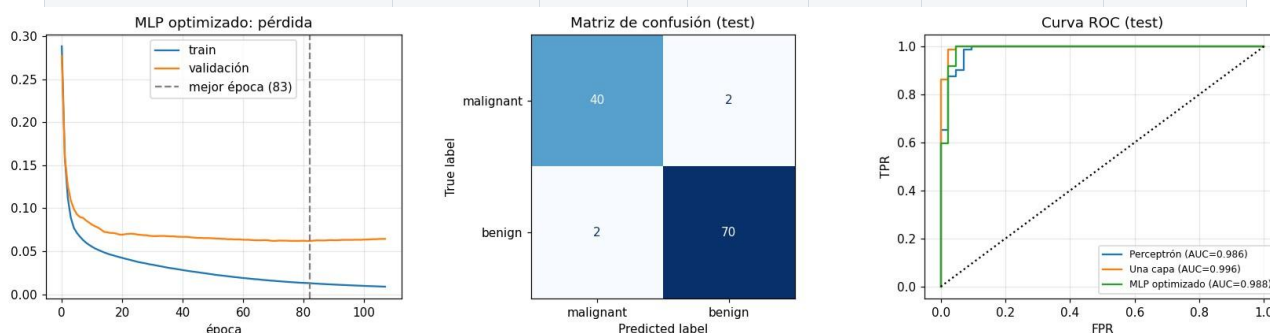


Figura 7. MLP optimizado: curvas de pérdida con la época de parada, matriz de confusión en test y curvas ROC de los tres modelos NumPy.

5.3 Análisis

- Todos los modelos superan el 93 % de exactitud y 0.98 de ROC-AUC. El problema es **casi linealmente separable**: la red de una capa (2 errores de 114) supera al MLP optimizado (4 errores). La diferencia equivale a 2 muestras, dentro de la variabilidad esperable con este tamaño de test; ante un desempeño equivalente se prefiere el modelo más simple.

- El perceptrón es el más débil (6 errores, recall de malignos 0.93): no minimiza una pérdida continua y no ofrece probabilidades calibradas.
- ReLU es mucho más **robusta a los hiperparámetros** (exactitud CV media 0.969 frente a 0.844). Con η bajo y L2 fuerte la red sigmoide queda atrapada en la predicción constante (exactitud mínima 0.63 = proporción de la clase mayoritaria).
- L2 = 5 resultó excesiva (ninguna configuración con ese valor aparece entre las 10 mejores); el control del sobreajuste lo aporta principalmente el early stopping.
- Elegir hiperparámetros con una única partición de validación (114 muestras) dio una configuración sobreajustada a esa partición; la validación cruzada de 5 folds produjo una selección más estable.
- La réplica en Keras de la misma arquitectura alcanza un ROC-AUC equivalente (0.992 frente a 0.988), lo que valida la implementación manual de backpropagation; las diferencias de exactitud provienen de la inicialización y de la parada temprana (77 épocas).

6. Clasificación multiclase en MNIST con TensorFlow + Keras

Cada imagen de 28×28 píxeles se aplanan en un vector de 784 valores normalizados a [0, 1] y se clasifica con un perceptrón multicapa totalmente conectado. Se reservan las últimas 6 000 imágenes de entrenamiento como validación (la misma partición se usa en PyTorch).

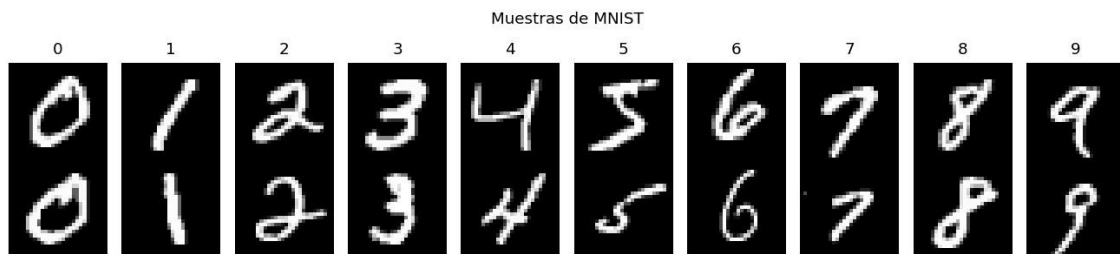


Figura 8. Muestras del dataset MNIST.

Tabla 12. Arquitectura MLP 784-256-128-10 (idéntica en Keras y PyTorch). Optimizador Adam ($\eta = 10^{-3}$), lote 128, early stopping con paciencia 3.

Capa	Tipo	Salida	Activación	Parámetros
Entrada	Vector aplanado	784	—	0
Oculto 1	Dense + Dropout(0.2)	256	ReLU	200 960
Oculto 2	Dense + Dropout(0.2)	128	ReLU	32 896
Salida	Dense	10	Softmax	1 290
Total				235 146

Código: carga y preparación de MNIST

```
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.mnist.load_data()
print("train:", x_train_full.shape, "test:", x_test.shape, "| rango píxeles:", x_train_full.min(), "-",
x_train_full.max())
def prep(x): return x.reshape(len(x), -1).astype("float32") / 255.0 # aplanar
28x28 -> 784 y normalizar

x_train, y_train = prep(x_train_full[:54000]), y_train_full[:54000]
x_val, y_val = prep(x_train_full[54000:]), y_train_full[54000:]
x_test_f = prep(x_test)
print("Formas finales:", x_train.shape, x_val.shape, x_test_f.shape)
```

Código: construcción del modelo en Keras

```
def build_mlp(activation="relu", dropout=0.2):
    layers = [keras.Input(shape=(784,))]
    for units in (256, 128):
        layers.append(keras.layers.Dense(units, activation=activation,
                                          kernel_initializer="he_normal" if activation == "relu" else
                                          "glorot_uniform"))
    if dropout: layers.append(keras.layers.Dropout(dropout))
    layers.append(keras.layers.Dense(10, activation="softmax"))
    model = keras.Sequential(layers, name=f"MLP_{activation}")
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model
```

Código: entrenamiento con EarlyStopping

```
keras.utils.set_random_seed(SEED)
model = build_mlp("relu",
                  dropout=0.2) model.summary()

callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)]
t0 = time.time()
history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=30,
                    batch_size=128, callbacks=callbacks, verbose=2) train_time = time.time() - t0
print(f"\nTiempo de entrenamiento: {train_time:.1f} s | épocas ejecutadas: {len(history.history['loss'])}")
```

6.1 Resultados

Tabla 13. Experimento Sigmoide vs ReLU en Keras (exactitud en validación, sin dropout).

Activación oculta	Época 1	Época 2	Época 3	Época 4	Época 5
Sigmoide	0.9358	0.9553	0.9630	0.9665	0.9702
ReLU	0.9672	0.9752	0.9772	0.9752	0.9733

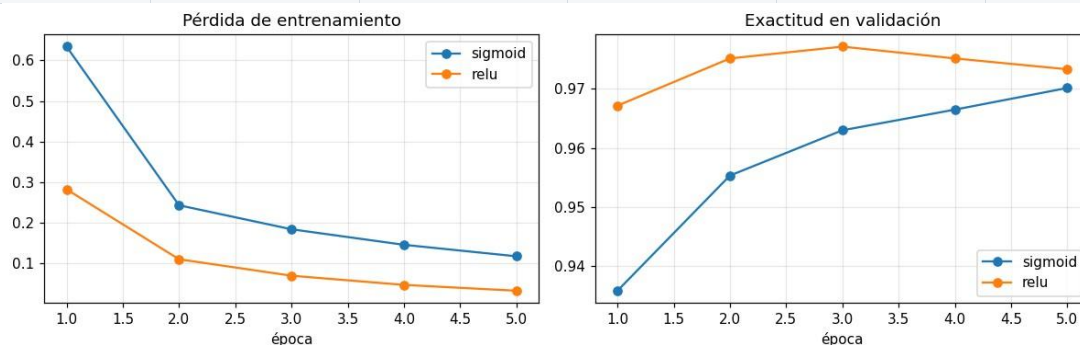


Figura 9. Keras: Sigmoide vs ReLU durante 5 épocas.

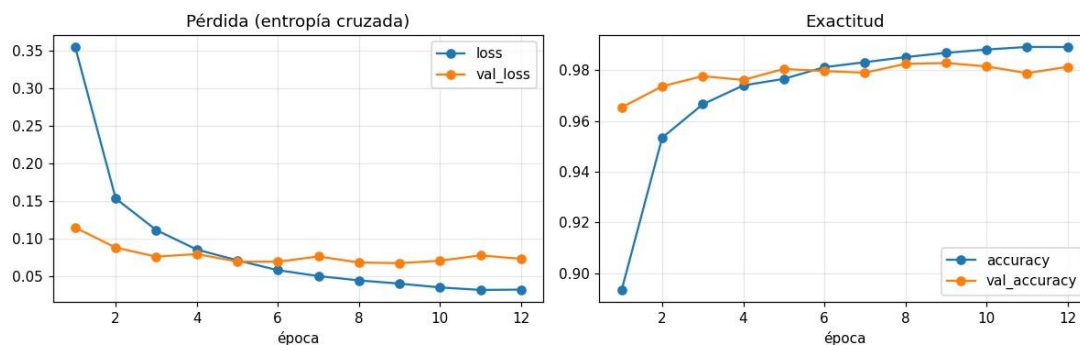


Figura 10. Curvas de entrenamiento del modelo final (12 épocas; early stopping restauró los pesos de la época 9).

El modelo final alcanzó una **exactitud en test de 98.22 %** (pérdida 0.0626; 178 errores de 10 000) en 24.6 s de entrenamiento en CPU. Las métricas por clase se sitúan entre 0.973 y 0.992 de F1; la clase más difícil es el 8 (precisión 0.958) y el 9 (recall 0.966).

Tabla 14. Reporte de clasificación en test — Keras.

Clase	0	1	2	3	4	5	6	7	8	9	Macro
-------	---	---	---	---	---	---	---	---	---	---	-------

Precisión	0.992	0.991	0.975	0.986	0.985	0.989	0.983	0.980	0.958	0.983	0.982
Recall	0.987	0.993	0.982	0.978	0.987	0.975	0.990	0.976	0.988	0.966	0.982
F1	0.989	0.992	0.978	0.982	0.986	0.982	0.987	0.978	0.973	0.975	0.982

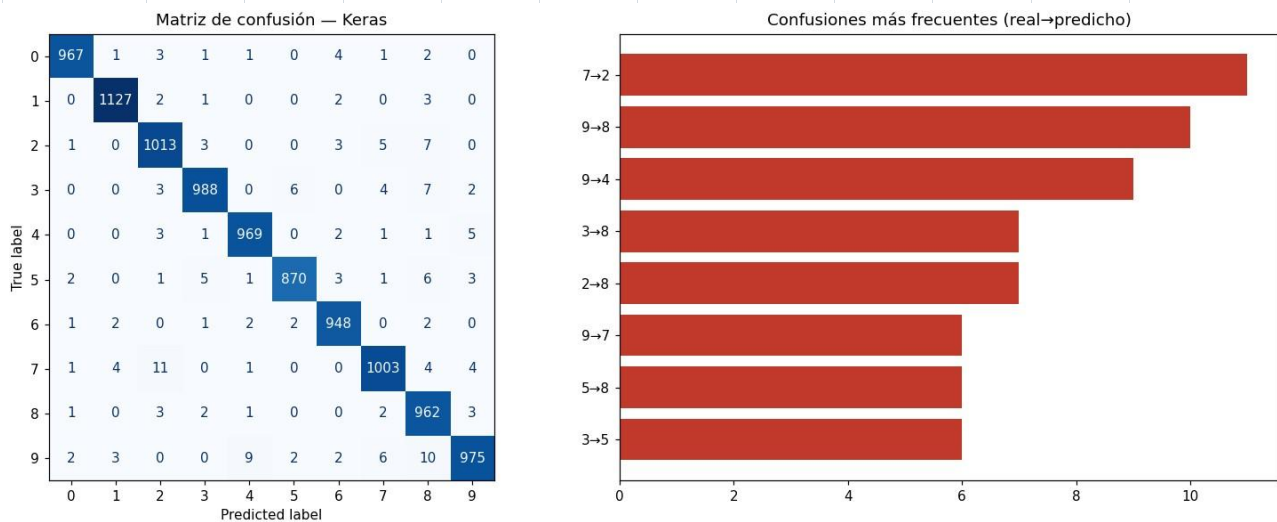


Figura 11. Matriz de confusión en test y confusiones más frecuentes (real→predicho).

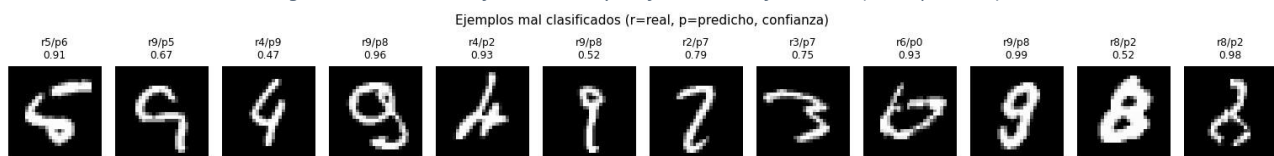


Figura 12. Ejemplos mal clasificados con la confianza del modelo.

Las confusiones más frecuentes (7→2, 9→8, 9→4, 3→8) corresponden a dígitos de trazo parecido. Un MLP trata cada píxel de forma independiente y no aprovecha la estructura espacial local de la imagen; esa es la limitación que resuelven las redes convolucionales, excluidas de esta actividad.

7. Clasificación multiclase en MNIST con PyTorch

Se replicó exactamente la arquitectura y los hiperparámetros del modelo Keras. La diferencia principal es que en PyTorch el bucle de entrenamiento es explícito: cada iteración ejecuta el paso hacia adelante, calcula la pérdida, llama a `loss.backward()` para obtener todos los gradientes por autograd (backpropagation) y actualiza los pesos con `optimizer.step()`. La última capa produce *logits* y la softmax queda integrada en `nn.CrossEntropyLoss`, que es numéricamente más estable.

Código: definición del modelo en PyTorch

```
class MLP(nn.Module):
    def __init__(self,
        activation="relu", dropout=0.2):
        super().__init__()
        act = nn.ReLU if activation == "relu" else nn.Sigmoid
        self.net = nn.Sequential(
            nn.Linear(784, 256), act(), nn.Dropout(dropout),
            nn.Linear(256, 128), act(), nn.Dropout(dropout),
            nn.Linear(128, 10),
            # logits
        )
        for m in self.net:
            if isinstance(m, nn.Linear):
                if activation == "relu": nn.init.kaiming_normal_(m.weight,
                    nonlinearity="relu")
                else: nn.init.xavier_uniform_(m.weight)
                nn.init.zeros_(m.bias)
        def forward(self, x): return self.net(x)

model = MLP().to(device)
print(model)
print("Parámetros entrenables:", sum(p.numel() for p in model.parameters() if p.requires_grad))
```

Código: bucle de entrenamiento explícito con early stopping

```

criterion = nn.CrossEntropyLoss()

def run_epoch(model, loader, optimizer=None):
    """Si hay optimizer entrena (forward + backward + step); si no, sólo
    evalúa."""
    train = optimizer is not None
    model.train(train)
    total_loss, correct, n = 0.0, 0, 0
    with torch.set_grad_enabled(train):
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)
            # forward
            # backward
            # actualización de Adam
            if train:
                optimizer.zero_grad()
                loss.backward()
            optimizer.step()
            total_loss += loss.item() * len(xb); correct += (logits.argmax(1) == yb).sum().item(); n += len(xb)
    return total_loss / n, correct / n

def fit(model, epochs=30, lr=1e-3, patience=3, verbose=True):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    hist = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}
    best_loss, best_state, wait = np.inf, None, 0
    for ep in range(1, epochs + 1):
        t = time.time()
        tl, ta = run_epoch(model, train_loader, optimizer)
        vl, va = run_epoch(model, val_loader)
        for k, v in zip(hist, (tl, ta, vl, va)): hist[k].append(v)
        if verbose:
            print(f"Época {ep:2d} | loss {tl:.4f} | acc {ta:.4f} | val_loss {vl:.4f} | val_acc {va:.4f} | {time.time()-t:.1f}s")
            if vl < best_loss:
                best_loss, best_state, wait = vl, copy.deepcopy(model.state_dict()), 0
        else:
            wait += 1
        if wait >= patience:
            if verbose: print(f"Early stopping: sin mejora en val_loss durante {patience} épocas.")
            break
    model.load_state_dict(best_state)
    return hist

```

7.1 Resultados y comparación entre frameworks

Tabla 15. Experimento Sigmoide vs ReLU en PyTorch (exactitud en validación, sin dropout).

Activación oculta	Época 1	Época 2	Época 3	Época 4	Época 5
Sigmoide	0.9317	0.9472	0.9600	0.9667	0.9692
ReLU	0.9670	0.9750	0.9755	0.9798	0.9793

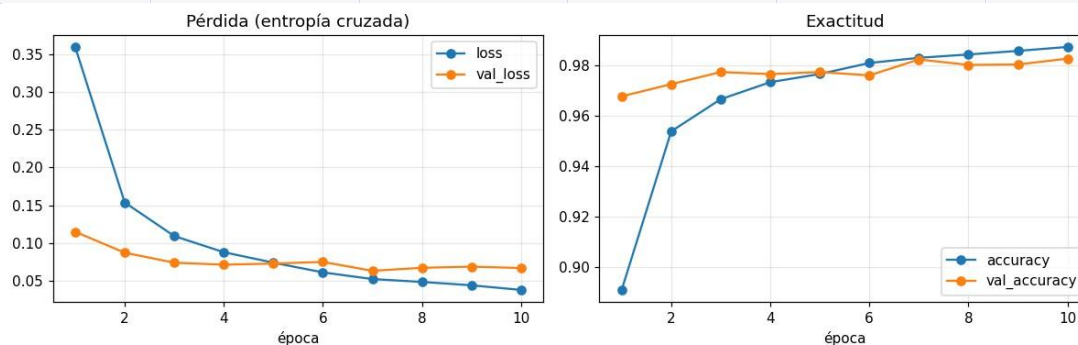


Figura 13. Curvas de entrenamiento del modelo PyTorch (10 épocas; mejores pesos de la época 7).

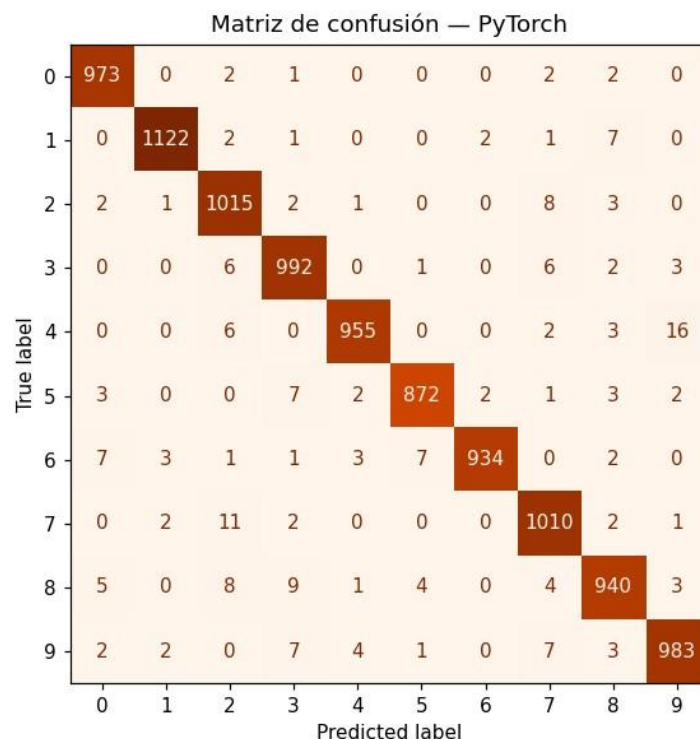


Figura 14. Matriz de confusión en test — PyTorch.

Tabla 16. Comparación Keras vs PyTorch en MNIST (test de 10 000 imágenes).

Framework	Parám.	Época s	Tiempo	Mejor val_acc	Acc. test	Pérdida test	Errores
TensorFlow + Keras 2.21	235 146	12	24.6 s	0.9828	0.9822	0.0626	178
PyTorch 2.14	235 146	10	16.6 s	0.9828	0.9796	0.0666	204

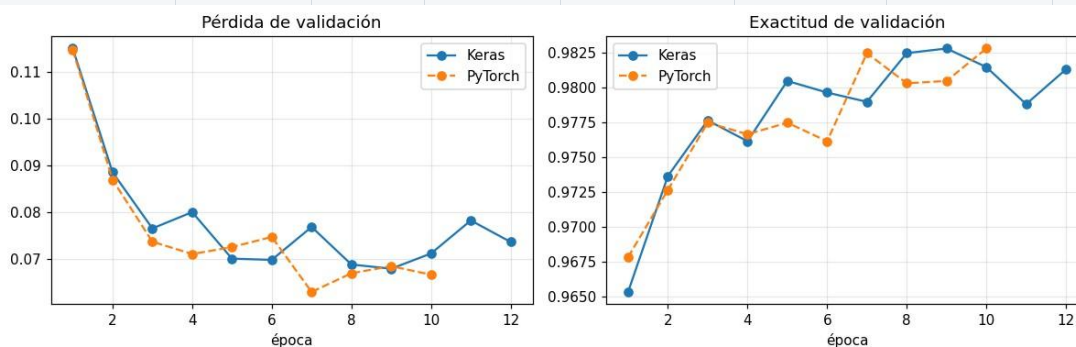


Figura 15. Pérdida y exactitud de validación por época en ambos frameworks.

Análisis. Con la misma arquitectura e hiperparámetros ambos frameworks alcanzan la misma exactitud máxima de validación (98.28 %) y una exactitud de test casi igual (98.22 % frente a 97.96 %, 0.26 puntos porcentuales). La diferencia se explica por la inicialización aleatoria, el orden de los mini-lotes y la época en que actúa el early stopping. Keras requiere menos código (compile + fit); PyTorch hace visible cada paso de backpropagation y da un control más fino del proceso. En ambos frameworks se reproduce el resultado del Notebook 01: **ReLU supera a Sigmoide** con igual número de épocas.

8. Evidencias de ejecución

Las siguientes capturas corresponden a celdas de los notebooks ejecutados de principio a fin (Jupyter, kernel Python 3), con su código y su salida. Los notebooks del repositorio se publican **con todas sus salidas**, de modo que las evidencias pueden verificarse directamente en GitHub o volver a generarse en Google Colab.

```
In [3]: class Perceptron:
        """Perceptrón de Rosenblatt con activación escalón."""
        def __init__(self, lr=0.1, epochs=50, seed=SEED):
            self.lr, self.epochs, self.rng = lr, epochs, np.random.default_rng(seed)

        def fit(self, X, y):
            self.w = self.rng.normal(0, 0.01, X.shape[1]); self.b = 0.0
            self.errors_ = []
            for _ in range(self.epochs):
                errors = 0
                for i in self.rng.permutation(len(X)):
                    update = self.lr * (y[i] - self.predict(X[i:i+1])[0])
                    self.w += update * X[i]; self.b += update
                    errors += int(update != 0)
                self.errors_.append(errors)
                if errors == 0: # convergió
                    break
            return self

        def decision_function(self, X): return X @ self.w + self.b
        def predict(self, X): return step(self.decision_function(X))
        def score(self, X, y): return (self.predict(X) == y).mean()

X_logic = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
TABLES = {"AND": np.array([0, 0, 0, 1]), "OR": np.array([0, 1, 1, 1]), "XOR": np.array([0, 1, 1, 0])}

for gate, y in TABLES.items():
    p = Perceptron(lr=0.1, epochs=100).fit(X_logic, y)
    print(f"{gate}>3: predicción={p.predict(X_logic).astype(int)} real={y.astype(int)} "
          f"exactitud={p.score(X_logic, y):.2f} épocas={len(p.errors_)}")

AND: predicción=[0 0 0 1] real=[0 0 0 1] exactitud=1.00 épocas=5
OR: predicción=[0 1 1 1] real=[0 1 1 1] exactitud=1.00 épocas=4
XOR: predicción=[0 0 0 0] real=[0 1 1 0] exactitud=0.50 épocas=100
```

Figura 16. Notebook 01 — perceptrón en AND, OR y XOR.

```
In [6]: def gradient_check(net, X, y, eps=1e-5):
        Y = net._prep_y(y)
        _, cache = net.forward(X)
        gW, _ = net.backward(Y, cache)
        analytic, numeric = [], []
        for l, W in enumerate(net.W):
            for idx in np.ndindex(W.shape):
                old = W[idx]
                W[idx] = old + eps; lp = net.loss(Y, net.forward(X)[0])
                W[idx] = old - eps; lm = net.loss(Y, net.forward(X)[0])
                W[idx] = old
                numeric.append((lp - lm) / (2 * eps)); analytic.append(gW[l][idx])
        a, n = np.array(analytic), np.array(numeric)
        return np.linalg.norm(a - n) / (np.linalg.norm(a) + np.linalg.norm(n))

rng = np.random.default_rng(0)
Xg = rng.normal(size=(20, 3))
for hidden in ["sigmoid", "tanh", "relu"]:
    net_b = MLP([3, 5, 4, 1], hidden=hidden, l2=0.1); yb = rng.integers(0, 2, 20)
    net_m = MLP([3, 5, 4, 3], hidden=hidden, l2=0.1); ym = rng.integers(0, 3, 20)
    print(f"{hidden}>7 | binaria: {gradient_check(net_b, Xg, yb):.2e} | multiclase (softmax): {gradient_check(net_m, Xg, ym):.2e}")

sigmoid | binaria: 8.95e-11 | multiclase (softmax): 1.15e-10
tanh | binaria: 7.01e-11 | multiclase (softmax): 1.85e-10
relu | binaria: 3.77e-11 | multiclase (softmax): 5.59e-11
```

Figura 17. Notebook 01 — gradient checking de backpropagation.

```
In [8]: Xm, ym = make_moons(n_samples=600, noise=0.2, random_state=SEED)
Xm_tr, Xm_te, ym_tr, ym_te = train_test_split(Xm, ym, test_size=0.3, random_state=SEED, stratify=ym)

models = {
    "Perceptrón": Perceptron(lr=0.1, epochs=100).fit(Xm_tr, ym_tr),
    "Una capa (sigmoide)": SingleLayerNet(lr=0.5, epochs=2000).fit(Xm_tr, ym_tr),
    "MLP oculta Sigmoide": MLP([2, 16, 16, 1], hidden="sigmoid", lr=0.05, epochs=300).fit(Xm_tr, ym_tr, Xm_te, ym_te),
    "MLP oculta ReLU": MLP([2, 16, 16, 1], hidden="relu", lr=0.05, epochs=300).fit(Xm_tr, ym_tr, Xm_te, ym_te),
}

for name, mdl in models.items():
    print(f"{name:<22} exactitud train={mdl.score(Xm_tr, ym_tr):.3f} test={mdl.score(Xm_te, ym_te):.3f}")

xx, yy = np.meshgrid(np.linspace(-1.8, 2.8, 300), np.linspace(-1.3, 1.8, 300))
grid = np.c_[xx.ravel(), yy.ravel()]
fig, axes = plt.subplots(1, 4, figsize=(17, 3.9))
for a, (name, mdl) in zip(axes, models.items()):
    a.contourf(xx, yy, mdl.predict(grid).reshape(xx.shape), alpha=0.3, cmap="coolwarm")
    a.scatter(Xm_te[:, 0], Xm_te[:, 1], c=ym_te, cmap="coolwarm", s=12, edgecolor="k", lw=0.3)
    a.set_title(f"{name}\ntest acc = {mdl.score(Xm_te, ym_te):.3f}")
plt.tight_layout(); plt.savefig("figures/01_fronteras_moons.png", bbox_inches="tight"); plt.show()
```

Perceptrón exactitud train=0.883 test=0.867
 Una capa (sigmoide) exactitud train=0.871 test=0.867
 MLP oculta Sigmoide exactitud train=0.962 test=0.967
 MLP oculta ReLU exactitud train=0.964 test=0.978

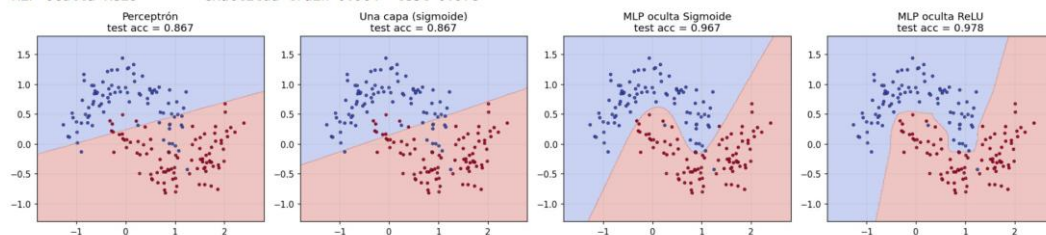


Figura 18. Notebook 01 — comparación de modelos en make_moons.

```
In [8]: class MLEarlyStopping(MLP):
def __init__(self, *args, patience=20, **kw):
    super().__init__(*args, **kw); self.patience = patience

def fit(self, X, y, X_val, y_val):
    Y, Yv = self._prep_y(y), self._prep_y(y_val)
    self.history = {"loss": [], "acc": [], "val_loss": [], "val_acc": []}
    best, wait = np.inf, 0
    for ep in range(self.epochs):
        idx = self.rng.permutation(len(X))
        for s in range(0, len(X), self.bs):
            bi = idx[s:s+self.bs]; A, cache = self.forward(X[bi]); self._step(*self.backward(Y[bi], cache))
        vl = self.loss(Yv, self.forward(X_val)[0])
        self.history["loss"].append(self.loss(Y, self.forward(X)[0])); self.history["acc"].append(self.score(X, y))
        self.history["val_loss"].append(vl); self.history["val_acc"].append(self.score(X_val, y_val))
        if vl < best - 1e-5:
            best, wait, self.best_epoch = vl, 0, ep + 1
            self._best = (lw.copy() for w in self.W), (b.copy() for b in self.b)
        else:
            wait += 1
            if wait >= self.patience: break
    self.W, self.b = self._best
    return self

grid = {"hidden_layers": [(16,), (32, 16), (64, 32)], "activation": ["sigmoid", "relu"],
        "lr": [0.005, 0.02], "l2": [0.0, 5.0]}

# Validación cruzada estratificada de 5 particiones sobre train+validación (455 muestras):
# una sola partición de validación (114 muestras) es demasiado ruidosa para elegir hiperparámetros.
X_dev, y_dev = np.vstack([X_tr, X_val]), np.concatenate([y_tr, y_val])
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
rows, t0 = [], time.time()
for hl, act, lr, l2 in itertools.product(*grid.values()):
    losses, accs, epochs = [], [], []
    for k, (i_tr, i_val) in enumerate(skf.split(X_dev, y_dev)):
        sc = StandardScaler().fit(X_dev[i_tr]) # escalador ajustado dentro de cada fold
        a, b = sc.transform(X_dev[i_tr]), sc.transform(X_dev[i_val])
        m = MLEarlyStopping([30, *hl, 1], hidden=act, lr=lr, l2=l2, epochs=400, batch_size=32,
                             patience=25, seed=SEED + k).fit(a, y_dev[i_tr], b, y_dev[i_val])
        losses.append(min(m.history["val_loss"])); accs.append(m.score(b, y_dev[i_val])); epochs.append(m.best_epoch)
    rows.append({"capas": str(hl), "activación": act, "lr": lr, "l2": l2, "épocas (media)": np.mean(epochs),
                "cv_loss": np.mean(losses), "cv_loss_std": np.std(losses), "cv_acc": np.mean(accs)})
search = pd.DataFrame(rows).sort_values("cv_loss").reset_index(drop=True)
print(f"{len(rows)} configuraciones x 5 folds = {5*len(rows)} entrenamientos en {time.time()-t0:.1f} s")
search.head(10).round(4)
```

24 configuraciones x 5 folds = 120 entrenamientos en 32.0 s

```
Out[8]:
```

	capas	activación	lr	l2	épocas (media)	cv_loss	cv_loss_std	cv_acc
0	(16,)	relu	0.020	0.0	102.0	0.0570	0.0351	0.9824
1	(64, 32)	sigmoid	0.020	0.0	121.0	0.0622	0.0378	0.9780
2	(64, 32)	relu	0.005	0.0	122.2	0.0645	0.0564	0.9780
3	(32, 16)	relu	0.020	0.0	28.4	0.0647	0.0394	0.9846
4	(64, 32)	sigmoid	0.005	0.0	236.6	0.0663	0.0360	0.9758
5	(64, 32)	relu	0.020	0.0	41.8	0.0665	0.0559	0.9802
6	(16,)	sigmoid	0.020	0.0	72.2	0.0672	0.0339	0.9758
7	(32, 16)	sigmoid	0.020	0.0	66.2	0.0673	0.0350	0.9780
8	(16,)	sigmoid	0.005	0.0	243.6	0.0680	0.0332	0.9780
9	(16,)	relu	0.005	0.0	125.0	0.0685	0.0284	0.9824

Figura 19. Notebook 02 — búsqueda de hiperparámetros con validación cruzada.

```
In [12]: import tensorflow as tf
from tensorflow import keras
tf.random.set_seed(SEED); keras.utils.set_random_seed(SEED)
init = "he_normal" if best_cfg["activación"] == "relu" else "glorot_normal"
# En la MLP NumPy el gradiente L2 por mini-lote es L2·w/batch → equivalente Keras: λ = L2 / (2·batch)
reg = keras.regularizers.l2(best_cfg["l2"] / (2 * 32)) if best_cfg["l2"] > 0 else None
keras_bin = keras.Sequential([keras.Input(shape=(30,)) +
    [keras.layers.Dense(n, activation=best_cfg["activación"], kernel_initializer=init, kernel_regularizer=reg) for n in hl]
    [keras.layers.Dense(1, activation="sigmoid")])
keras_bin.compile(optimizer=keras.optimizers.SGD(learning_rate=best_cfg["lr"], momentum=0.9),
    loss="binary_crossentropy", metrics=["accuracy", keras.metrics.AUC(name="auc")])
keras_bin.summary()
hist = keras_bin.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=400, batch_size=32, verbose=0,
    callbacks=[keras.callbacks.EarlyStopping(patience=25, restore_best_weights=True)])
pk = keras_bin.predict(X_te, verbose=0)[: , 0]
final_df.loc["Keras (misma arquitectura)"] = pd.Series(metrics(y_te, (pk >= 0.5).astype(float), pk)).round(4)
print("Épocas ejecutadas en Keras:", len(hist.history["loss"]))
final_df
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	496
dense_1 (Dense)	(None, 1)	17

Total params: 513 (2.00 KB)

Trainable params: 513 (2.00 KB)

Non-trainable params: 0 (0.00 B)

Épocas ejecutadas en Keras: 77

	Exactitud	Precisión	Recall	F1	Recall malignos	ROC-AUC
Perceptrón	0.9474	0.9583	0.9583	0.9583	0.9286	0.9861
Red de una capa	0.9825	0.9861	0.9861	0.9861	0.9762	0.9964
MLP optimizado (16,) relu	0.9649	0.9722	0.9722	0.9722	0.9524	0.9884
Keras (misma arquitectura)	0.9386	0.9851	0.9167	0.9496	0.9762	0.9924

Figura 20. Notebook 02 — réplica en Keras y tabla final en test.

```
In [4]: keras.utils.set_random_seed(SEED)
model = build_mlp("relu", dropout=0.2)
model.summary()

callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)]
t0 = time.time()
history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=30, batch_size=128,
    callbacks=callbacks, verbose=2)
train_time = time.time() - t0
print(f"\nTiempo de entrenamiento: {train_time:.1f} s | épocas ejecutadas: {len(history.history['loss'])}")
```

Model: "MLP_relu"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 256)	200,960
dropout (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 10)	1,290

Total params: 235,146 (918.54 KB)

Trainable params: 235,146 (918.54 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/30

422/422 - 3s - 7ms/step - accuracy: 0.8934 - loss: 0.3547 - val_accuracy: 0.9653 - val_loss: 0.1151

Epoch 2/30

422/422 - 2s - 4ms/step - accuracy: 0.9534 - loss: 0.1537 - val_accuracy: 0.9737 - val_loss: 0.0886

Epoch 3/30

422/422 - 2s - 5ms/step - accuracy: 0.9666 - loss: 0.1118 - val_accuracy: 0.9777 - val_loss: 0.0765

Epoch 4/30

422/422 - 2s - 4ms/step - accuracy: 0.9740 - loss: 0.0860 - val_accuracy: 0.9762 - val_loss: 0.0800

Epoch 5/30

422/422 - 2s - 5ms/step - accuracy: 0.9766 - loss: 0.0717 - val_accuracy: 0.9805 - val_loss: 0.0700

Epoch 6/30

422/422 - 2s - 4ms/step - accuracy: 0.9812 - loss: 0.0586 - val_accuracy: 0.9797 - val_loss: 0.0698

Epoch 7/30

422/422 - 2s - 4ms/step - accuracy: 0.9831 - loss: 0.0508 - val_accuracy: 0.9790 - val_loss: 0.0768

Epoch 8/30

422/422 - 3s - 6ms/step - accuracy: 0.9852 - loss: 0.0451 - val_accuracy: 0.9825 - val_loss: 0.0688

Epoch 9/30

422/422 - 2s - 4ms/step - accuracy: 0.9868 - loss: 0.0408 - val_accuracy: 0.9828 - val_loss: 0.0679

Epoch 10/30

422/422 - 2s - 4ms/step - accuracy: 0.9881 - loss: 0.0358 - val_accuracy: 0.9815 - val_loss: 0.0711

Epoch 11/30

422/422 - 2s - 4ms/step - accuracy: 0.9892 - loss: 0.0324 - val_accuracy: 0.9788 - val_loss: 0.0782

Epoch 12/30

422/422 - 2s - 5ms/step - accuracy: 0.9891 - loss: 0.0328 - val_accuracy: 0.9813 - val_loss: 0.0736

Tiempo de entrenamiento: 24.6 s | épocas ejecutadas: 12

Figura 21. Notebook 03 — resumen del modelo y entrenamiento en Keras.

```
In [6]: test_loss, test_acc = model.evaluate(x_test_f, y_test, verbose=0)
probs = model.predict(x_test_f, verbose=0)
y_pred = probs.argmax(axis=1)
print(f'Pérdida test = {test_loss:.4f} | Exactitud test = {test_acc:.4f} | Errores = {(y_pred != y_test).sum()} de {len(y_test)}')
print(classification_report(y_test, y_pred, digits=4))
```

Pérdida test = 0.0626 | Exactitud test = 0.9822 | Errores = 178 de 10000

	precision	recall	f1-score	support
0	0.9918	0.9867	0.9893	980
1	0.9912	0.9930	0.9921	1135
2	0.9750	0.9816	0.9783	1032
3	0.9860	0.9782	0.9821	1010
4	0.9848	0.9868	0.9858	982
5	0.9886	0.9753	0.9819	892
6	0.9834	0.9896	0.9865	958
7	0.9804	0.9757	0.9781	1028
8	0.9582	0.9877	0.9727	974
9	0.9829	0.9663	0.9745	1009
accuracy			0.9822	10000
macro avg	0.9822	0.9821	0.9821	10000
weighted avg	0.9823	0.9822	0.9822	10000

Figura 22. Notebook 03 — evaluación en test (Keras).

```
In [6]: torch.manual_seed(SEED)
model = MLP("relu", dropout=0.2).to(device)
t0 = time.time()
history = fit(model, epochs=30, lr=1e-3, patience=3)
train_time = time.time() - t0
print(f'\nTiempo de entrenamiento: {train_time:.1f} s | épocas ejecutadas: {len(history['loss'])}')

h = pd.DataFrame(history); h.index += 1
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
h[["loss", "val_loss"]].plot(ax=ax[0], marker="o", title="Pérdida (entropía cruzada)")
h[["accuracy", "val_accuracy"]].plot(ax=ax[1], marker="o", title="Exactitud")
for a in ax: a.set_xlabel("época"); a.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("figures/04_curvas_pytorch.png", bbox_inches="tight"); plt.show()
```

Época 1 | loss 0.3602 | acc 0.8908 | val_loss 0.1148 | val_acc 0.9678 | 1.6s
 Época 2 | loss 0.1537 | acc 0.9539 | val_loss 0.0870 | val_acc 0.9727 | 1.6s
 Época 3 | loss 0.1090 | acc 0.9667 | val_loss 0.0737 | val_acc 0.9775 | 1.6s
 Época 4 | loss 0.0876 | acc 0.9735 | val_loss 0.0710 | val_acc 0.9767 | 1.6s
 Época 5 | loss 0.0737 | acc 0.9767 | val_loss 0.0725 | val_acc 0.9775 | 1.5s
 Época 6 | loss 0.0608 | acc 0.9811 | val_loss 0.0747 | val_acc 0.9762 | 1.6s
 Época 7 | loss 0.0519 | acc 0.9832 | val_loss 0.0629 | val_acc 0.9825 | 1.8s
 Época 8 | loss 0.0481 | acc 0.9845 | val_loss 0.0669 | val_acc 0.9803 | 1.9s
 Época 9 | loss 0.0437 | acc 0.9859 | val_loss 0.0684 | val_acc 0.9805 | 1.8s
 Época 10 | loss 0.0375 | acc 0.9875 | val_loss 0.0666 | val_acc 0.9828 | 1.8s
 Early stopping: sin mejora en val_loss durante 3 épocas.

Tiempo de entrenamiento: 16.6 s | épocas ejecutadas: 10

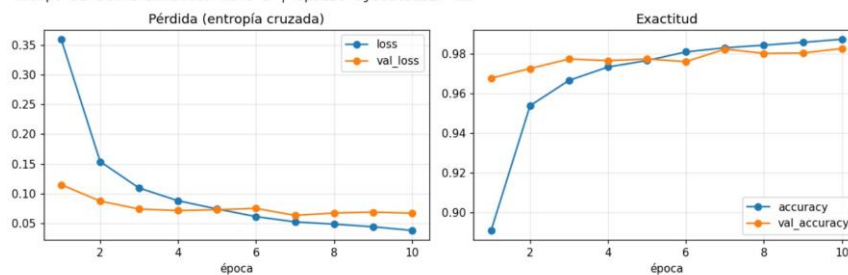


Figura 23. Notebook 04 — entrenamiento en PyTorch con early stopping.

```
In [9]: if os.path.exists("results/keras_mnist.json"):
kr = json.load(open("results/keras_mnist.json"))
cols = ["framework", "version", "params", "epochs", "train_time_s", "val_accuracy_best", "test_accuracy", "test_loss", "errors"]
comp = pd.DataFrame([{"c": kr[c] for c in cols}, {"c": results[c] for c in cols}]).set_index("framework")
display(comp.round(4)) if "display" in dir() else print(comp.round(4))
comp.to_csv("results/comparacion_frameworks.csv")

fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
for name, hh, st in [("Keras", kr["history"], "-"), ("PyTorch", history, "--")]:
    e = range(1, len(hh["val_loss"]) + 1)
    ax[0].plot(e, hh["val_loss"], st, marker="o", label=name); ax[1].plot(e, hh["val_accuracy"], st, marker="o", label=name)
ax[0].set_title("Pérdida de validación"); ax[1].set_title("Exactitud de validación")
for a in ax: a.set_xlabel("época"); a.legend(); a.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("figures/04_keras_vs_pytorch.png", bbox_inches="tight"); plt.show()
else:
    print("No se encontró results/keras_mnist.json: ejecute primero el Notebook 03.")
```

framework	version	params	epochs	train_time_s \
TensorFlow + Keras	2.21.0	235146	12	24.6
PyTorch	2.14.0+cu130	235146	10	16.6

framework	val_accuracy_best	test_accuracy	test_loss	errors
TensorFlow + Keras	0.9828	0.9822	0.0626	178
PyTorch	0.9828	0.9796	0.0666	204

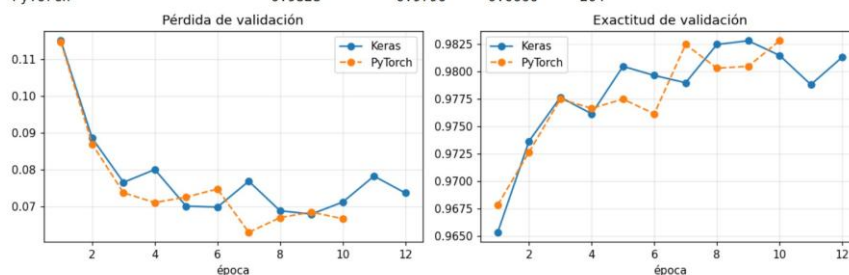


Figura 24. Notebook 04 — comparación Keras vs PyTorch.

9. Análisis comparativo y conclusiones

Tabla 17. Resumen de resultados.

Problema	Mejor modelo	Resultado en test	Idea clave
XOR	MLP 2-4-1	Exactitud 1.00	Se necesitan capas ocultas no lineales
make_moons	MLP 2-16-16-1 ReLU	Exactitud 0.978	ReLU converge ~8× más rápido que Sigmoide
Breast Cancer (binaria)	Red de una capa	Exactitud 0.982 · AUC 0.996	Datos casi lineales: el modelo simple basta
MNIST (multiclase)	MLP 784-256-128-10 (Keras)	Exactitud 0.982	Keras y PyTorch equivalentes (± 0.3 pp)

- **Backpropagation** se implementó desde cero y se verificó numéricamente (error relativo $\sim 10^{-10}$) para salidas binaria y multiclase y tres activaciones ocultas; los modelos equivalentes en Keras obtienen métricas comparables, lo que confirma la corrección de la implementación manual.
- El **perceptrón** y la **red de una capa** son clasificadores lineales: resuelven AND/OR y problemas casi lineales, pero no XOR ni make_moons. El **MLP** supera esa limitación gracias a las capas ocultas no lineales.
- **ReLU** fue consistentemente superior a **Sigmoide** en capas ocultas en los tres escenarios (convergencia más rápida y mayor robustez a los hiperparámetros), en coherencia con el análisis de las derivadas. La sigmoide sigue siendo la elección adecuada para la salida binaria y la softmax para la multiclase.
- El **entrenamiento supervisado** riguroso (particiones separadas, escalado ajustado sólo con train, validación cruzada, early stopping y evaluación única en test) fue tan importante como la arquitectura: la selección con una única partición de validación pequeña condujo a sobreajuste.
- En **MNIST**, un MLP sin convoluciones alcanza $\sim 98\%$ de exactitud. **TensorFlow + Keras** ofrece mayor productividad y **PyTorch** mayor transparencia y control; con la misma configuración producen resultados prácticamente idénticos.
- Como trabajo futuro: ajustar el umbral de decisión según el coste de los falsos negativos en el problema clínico, y explorar regularización adicional (BatchNorm, *data augmentation*) en MNIST dentro de arquitecturas densas.

10. Repositorio GitHub

Todo el código fuente, los notebooks ejecutados, las figuras y los resultados están publicados en: <https://github.com/jahirravila/redes-neuronales-II>

Tabla 18. Enlaces del repositorio y apertura directa de cada notebook en Google Colab.

Recurso	Enlace
Repositorio	https://github.com/jahirravila/redes-neuronales-II
Notebook 01 en Colab	01_perceptron_capas_backprop.ipynb
Notebook 02 en Colab	02_clasificacion_binaria_breast_cancer.ipynb
Notebook 03 en Colab	03_mnist_tensorflow_keras.ipynb
Notebook 04 en Colab	04_mnist_pytorch.ipynb

Código: estructura del repositorio

```
redes-neuronales-ii/
├── README.md
├── requirements.txt
├── notebooks/
│   ├── 01_perceptron_capas_backprop.ipynb
│   ├── 02_clasificacion_binaria_breast_cancer.ipynb
│   ├── 03_mnist_tensorflow_keras.ipynb
│   └── 04_mnist_pytorch.ipynb
├── figures/      (gráficas generadas)
└── results/     (métricas json/csv y modelos .keras / .pt)
```

```
|— src/
|   |— nn_numpy.py  (Perceptrón, SingleLayerNet, MLP)
|   |— docs/
|       |— Redes_Neuronales_II.pdf
```

Referencias

- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
- Minsky, M., & Papert, S. (1969). *Perceptrons*. MIT Press.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers. *ICCV*.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.
- Srivastava, N., et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15, 1929–1958.
- LeCun, Y., Cortes, C., & Burges, C. J. C. The MNIST database of handwritten digits.
- Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Documentación oficial: TensorFlow/Keras ([tensorflow.org](https://www.tensorflow.org), keras.io) y PyTorch (pytorch.org).